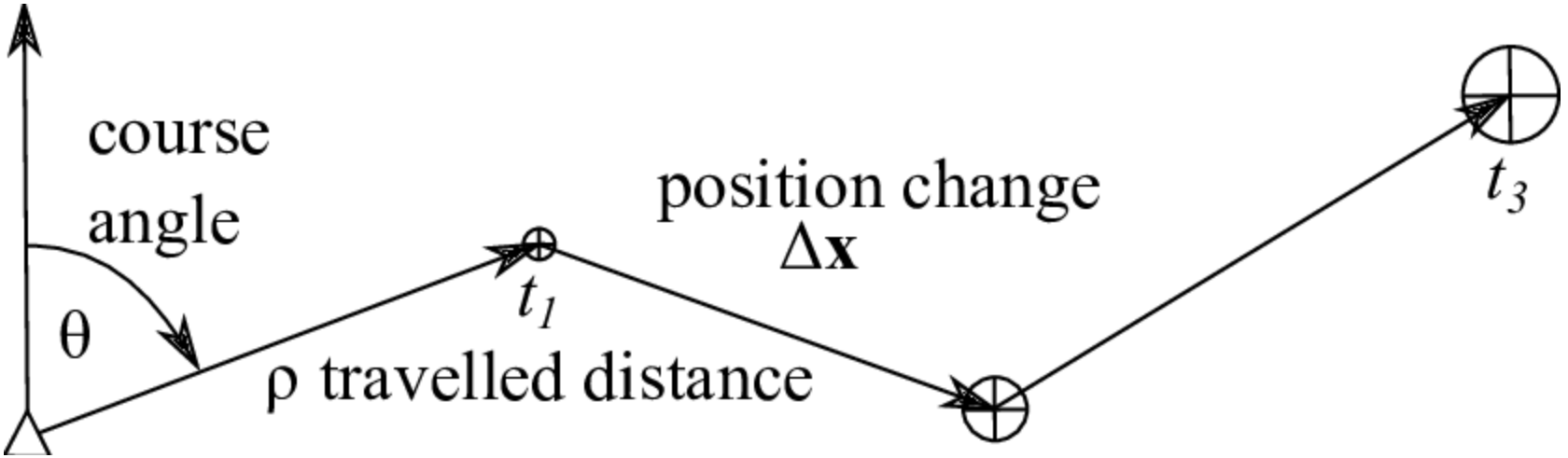
Improving Dead Reckoning

Prof Muthukumar

Dead Reckoning

- Dead Reckoning refers to estimating the robot's position based on its known starting point and the movement information collected over time, such as velocity, orientation, and odometry data.

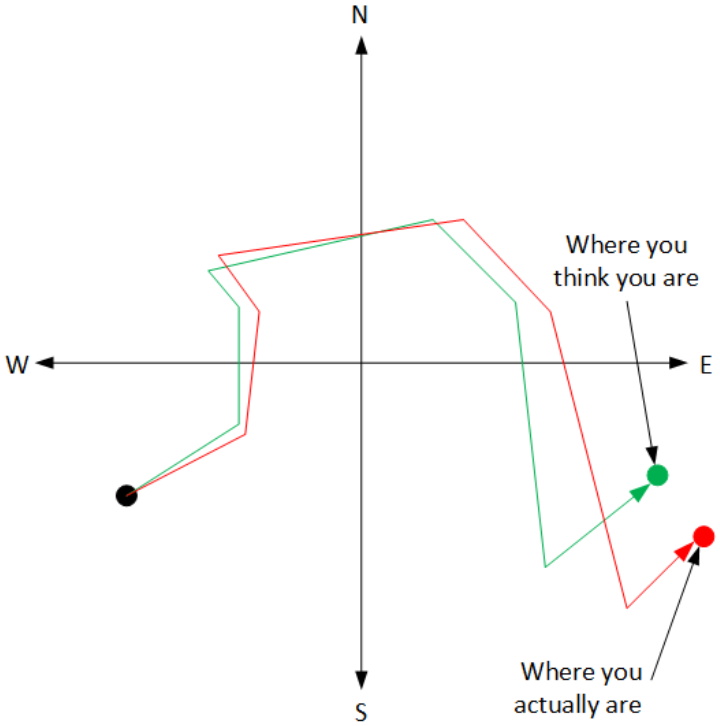


Dead Reckoning

- Also called deduced reckoning
- The process of (incrementally, at discrete time steps /Pose2D) determining its own position based on the knowledge of some reference point (a fix - /tf) and the knowledge or the estimate of the velocities (speeds and headings - /cmd_vel) actuated over time. Data related to exogenous and endogenous disturbances can be included.

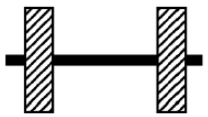
Incrementally build the state using on-board information

- On-board information to include: Odometry, IMU, GPS, Camera, etc.

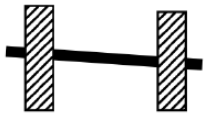


Dead Reckoning using Odometry

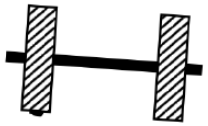
- Use wheel sensors to update position
- Odometry Error Sources?
  - Unequal wheel diameter
  - Variation in the contact point of the wheel
  - Unequal floor contact and variable friction can lead to slipping



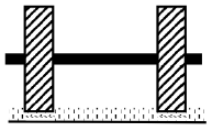
ideal case



different wheel diameters

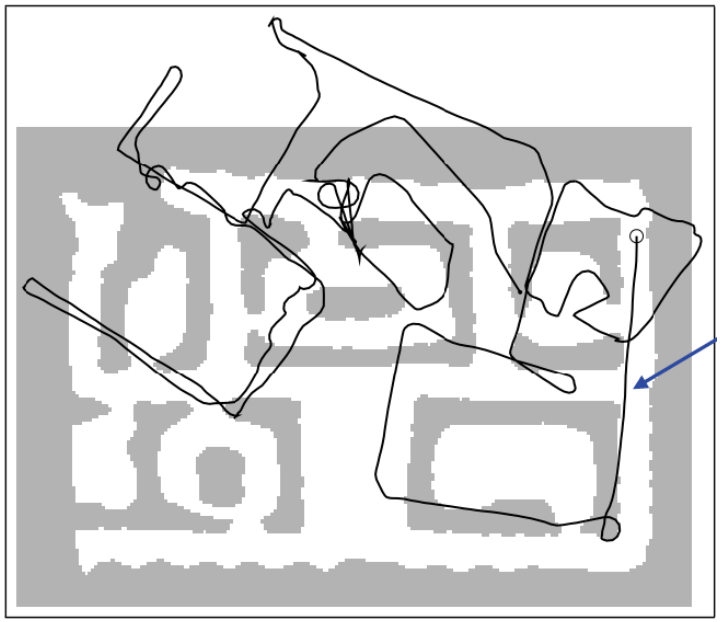


bump

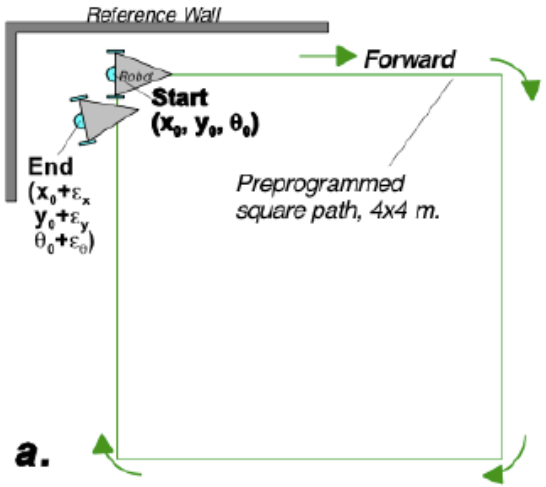


carpet

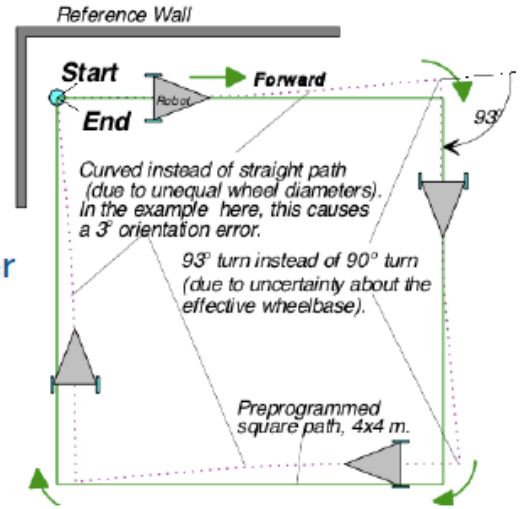
Result



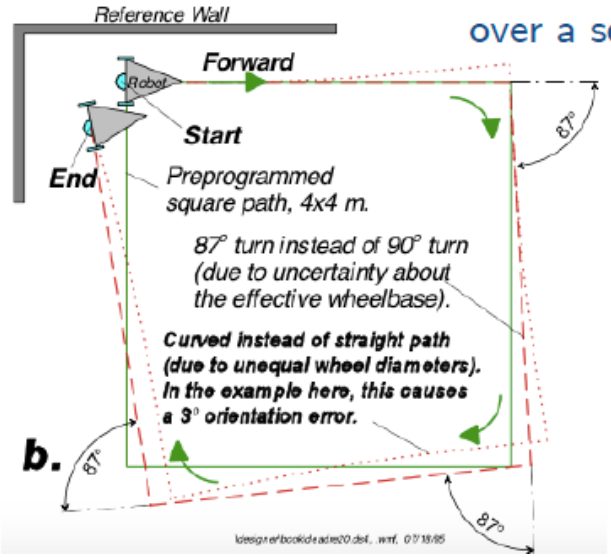
Common Errors = used for error modeling



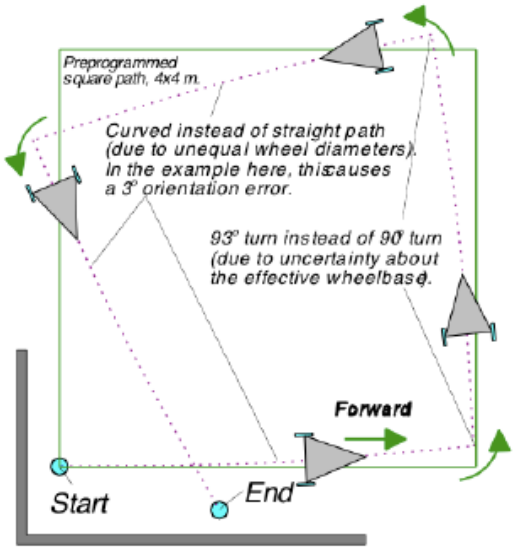
Different errors
canceling with each other



Cumulative error
over a squared path

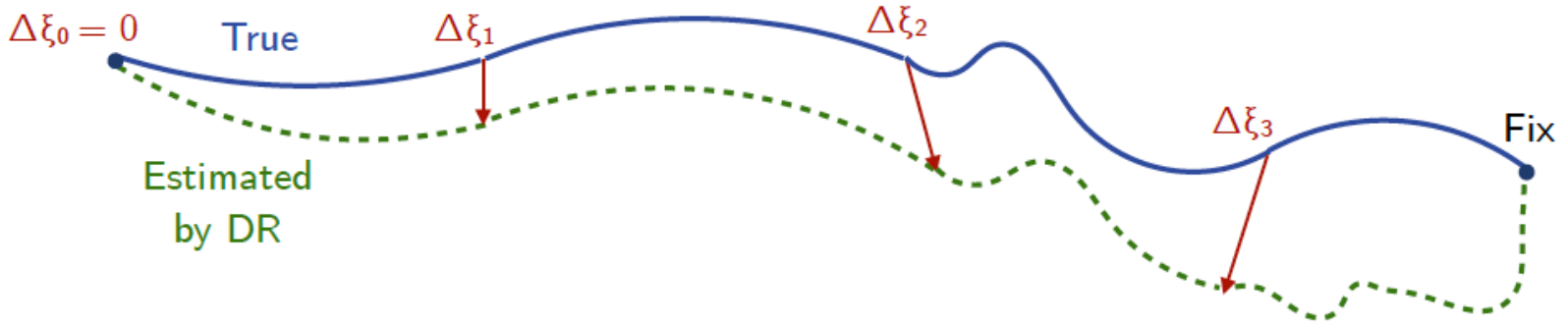


Different errors
summing up



Error Model

- Why model odometry errors? Incorporate odometric errors in its kinematics and dynamics.
- The movement of the robot, sensed with wheel encoders and/or heading sensors is integrated to compute position. Because the sensor measurement errors are integrated, the position error accumulates over time.
- We will see “Error Model for Odometric Position Estimation” for a Differential Drive Robot.



Error Model

basic discrete motion equations

$$\dot{x} = \frac{r}{2}(\dot{\phi}_1 + \dot{\phi}_2)\cos(\theta)$$

$$\dot{y} = \frac{r}{2}(\dot{\phi}_1 + \dot{\phi}_2)\sin(\theta)$$

$$\dot{\theta} = \frac{r}{2L}(\dot{\phi}_1 - \dot{\phi}_2)$$

$$x_{k+1} = x_k + \frac{r\Delta t}{2}(\omega_{1,k} + \omega_{2,k})\cos(\theta_k)$$

$$y_{k+1} = y_k + \frac{r\Delta t}{2}(\omega_{1,k} + \omega_{2,k})\sin(\theta_k)$$

$$\theta_{k+1} = \theta_k + \frac{r\Delta t}{2L}(\omega_{1,k} - \omega_{2,k})$$

Noise can be injected directly into the state variables (x, y, θ):

$$x_{k+1} = x_k + \frac{r\Delta t}{2}(\omega_{1,k} + \omega_{2,k})\cos(\theta_k) + \epsilon_1$$

$$y_{k+1} = y_k + \frac{r\Delta t}{2}(\omega_{1,k} + \omega_{2,k})\sin(\theta_k) + \epsilon_2$$

$$\theta_{k+1} = \theta_k + \frac{r\Delta t}{2L}(\omega_{1,k} - \omega_{2,k}) + \epsilon_3$$

Assume that we have random values (or vector) ϵi, i = 1, 2, 3 drawn from some normal distribution $N(\mu, \sigma)$.

If we inject the noise into the ω terms

$$x_{k+1} = x_k + \frac{r\Delta t}{2}(\omega_{1,k} + \omega_{2,k} + \epsilon_1)\cos(\theta_k)$$

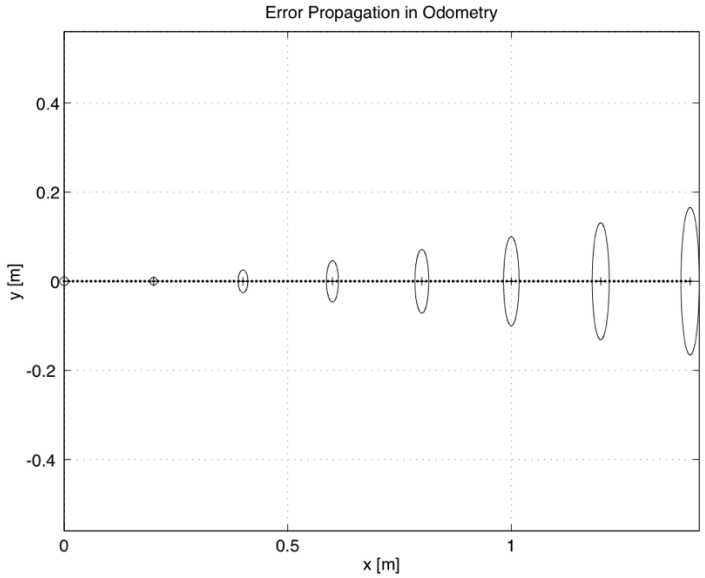
$$y_{k+1} = y_k + \frac{r\Delta t}{2}(\omega_{1,k} + \omega_{2,k} + \epsilon_1)\sin(\theta_k)$$

$$\theta_{k+1} = \theta_k + \frac{r\Delta t}{2L}(\omega_{1,k} - \omega_{2,k} + \epsilon_2)$$

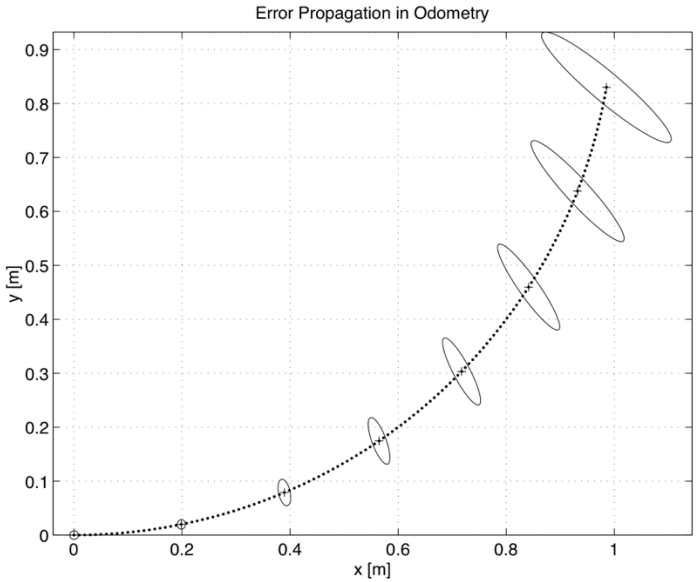
Offsetting Errors

- Once the error model has been established, the error parameters must be specified.
- One can compensate for deterministic errors properly calibrating the robot.
- Error parameters specifying the non-deterministic errors can only be quantified by statistical (repetitive) measurements.

Growth of the pose uncertainty for straight line movement



Growth of the pose uncertainty for circular movement

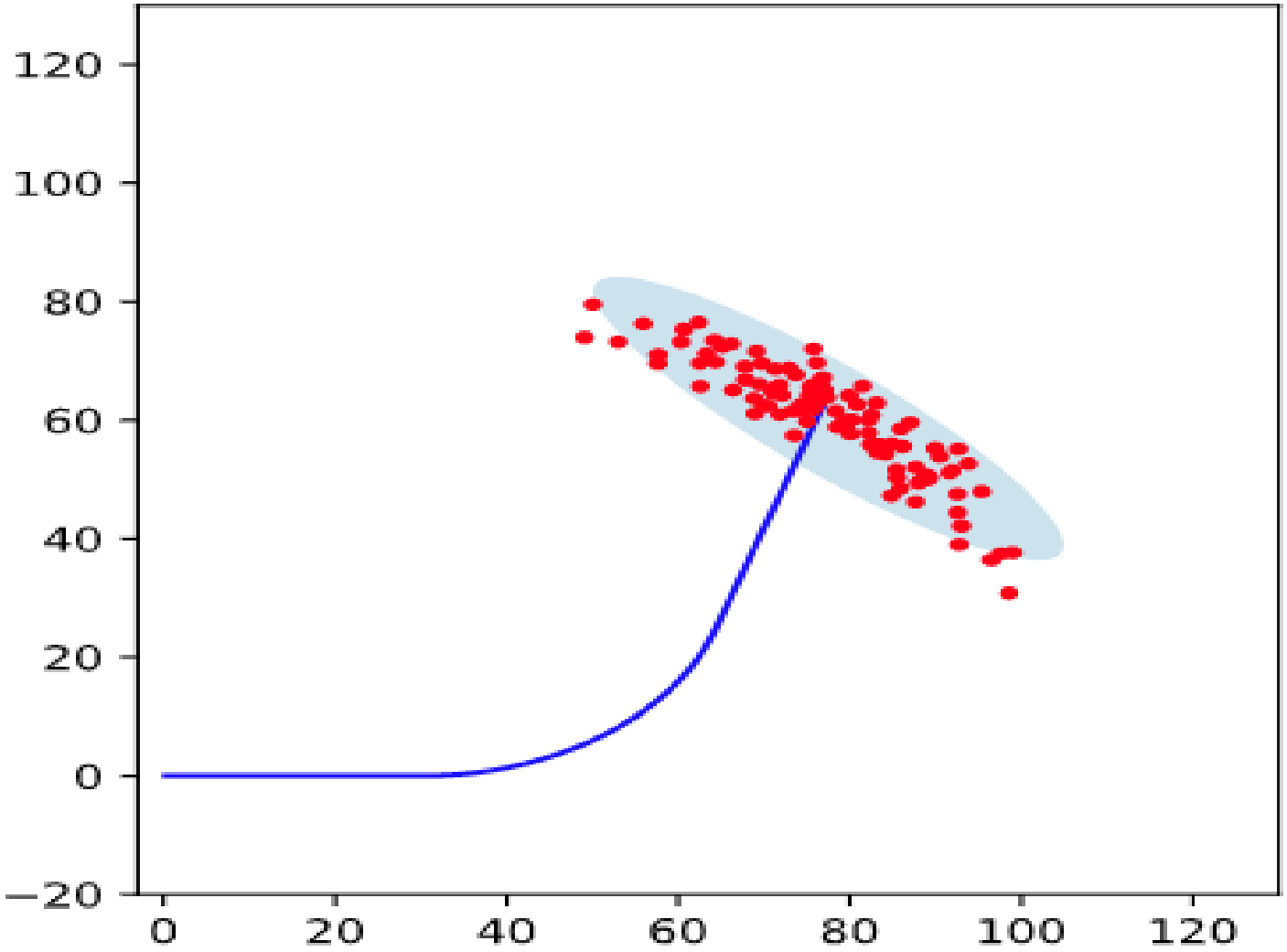


simulate_noise.py

```
for k in range(N):
    thv = 0.0
    xv = 0.0
    yv = 0.0
    i = 0
    xerr = np.random.normal(mu1,sigma1, NumP)
    yerr = np.random.normal(mu1,sigma1, NumP)
    therr = np.random.normal(mu2,sigma2, NumP)
    for t in tp:
      w1,w2 = wheels(t)
      dx = (r*dt/2.0)*(w1+w2)*cos(thv) + xerr[i]
      dy = (r*dt/2.0)*(w1+w2)*sin(thv) + yerr[i]
      dth = (r*dt/(2.0*l))*(w1-w2) + therr[i]
      xv = xv + dx
      yv = yv + dy
      thv = thv + dth
      xpath[k][i] = xv
      ypath[k][i] = yv
      thpath[k][i] = thv
      i = i+1
```

https://github.com/deeksha777/roboscience/tree/master/PathTracking/noiseSimulation

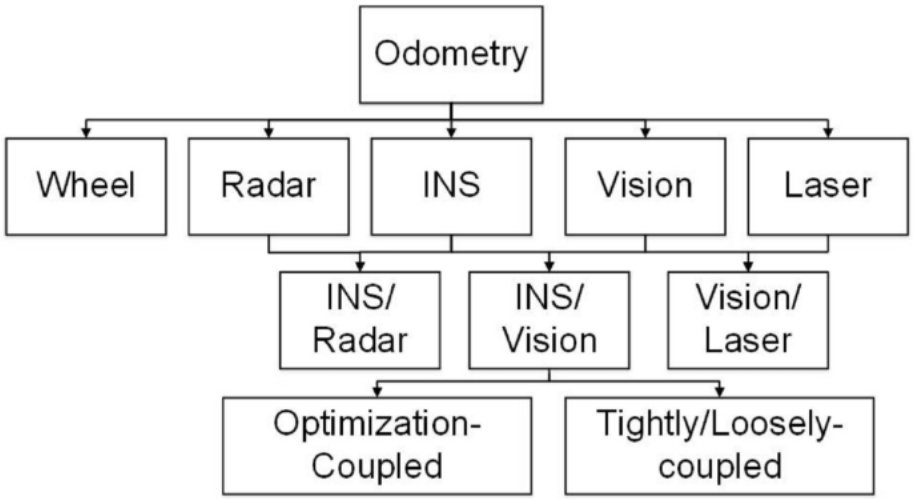
Simulation



Solutions

- Dead Reckoning
  - Using odometry data
  - Wheel slip, skid
- Dead Reckoning
  - Use IMU data with Odometry data
  - IMU Data
    - Need calibration and noisy
    - Has drift and computation accuracy issues
  - Noisy – Needs filtering
  - Filtering
    - High and low pass filtering
    - Complementary Filtering
    - Kalman Filtering
- Solution
  - Sensor Fusion
  - Optical Flow, Visual

Sensor Fusion

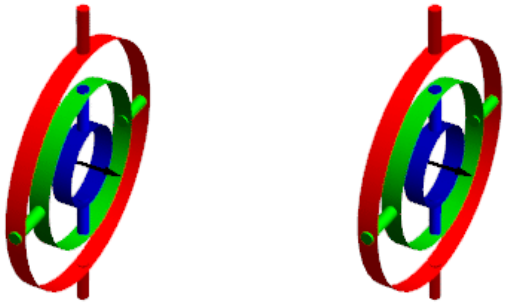


Euler angles

- The Euler angles are three angles introduced by Leonhard Euler to describe the orientation of a rigid body with respect to a fixed coordinate system.
- Euler angles suffer from being complicated at the code level - they require that an order of rotation is stored
- Gimbal lock is the loss of one degree of freedom in a three-dimensional system when two of the three gimbals become aligned, essentially causing a loss

$$R_{zyx}(a,b,c) = R_x(a)R_y(\pi/2)R_z(b)$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos a & -\sin a \\ 0 & \sin a & \cos a \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \cos c & -\sin c & 0 \\ \sin c & \cos c & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Quaternion

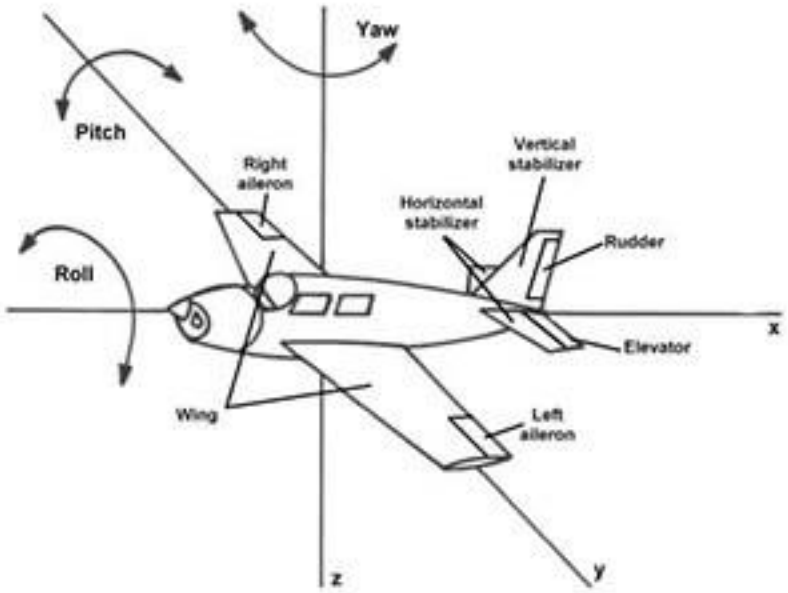
- Quaternions are an alternate way to describe orientation or rotations in 3D space using an ordered set of four numbers. They have the ability to uniquely describe any three-dimensional rotation about an arbitrary axis and do not suffer from gimbal lock.
- A quaternion is typically represented as:

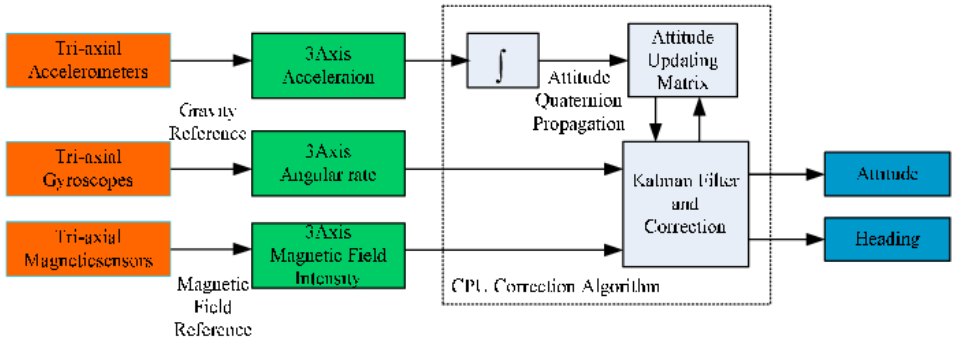
$$q = w + xi + yj + zk$$

  - "q" is the quaternion itself.
  - "w" is the scalar (real) part of the quaternion.
  - "x," "y," and "z" are the vector (imaginary) parts of the quaternion, each associated with one of the unit vectors i, j, and k, respectively.

Attitude and Heading Reference System (AHRS)

- An attitude and heading reference system (AHRS) consists of sensors on three axes that provide attitude information for robot, including roll, pitch and yaw.





Attitude and Heading Reference System (AHRS)

- Madgwick and Mahony with a basic fusion approach
- Madgwick filter represents all mean zero gyroscope measurement errors (β)
- Mahony filter takes into consideration the disparity between the orientation from the gyroscope and the estimation from the magnetometer and accelerometer and weighs them according to its gains.
- Changes made to the gyroscope are given by:

$$k_p\, e_m + k_i\, e_i \qquad e_i = e_m \frac{1}{f_s}$$

where k_p is the proportional gain, e_m is the sensor error of the gyroscope, k_i is the integral gain, e_i is the integral error, and f_s is the sampling frequency.

- basic filter uses a simpler weighting process directly on the quaternions.

$$\boldsymbol{\gamma}\, q_G + (1 - \boldsymbol{\gamma}) q_{AM}$$

where $\boldsymbol{\gamma}$ is the weighing coefficient, q_G is the quaternion of the gyroscope, and q_{AM} is the quaternion based on the acceleration and magnetometer readings.

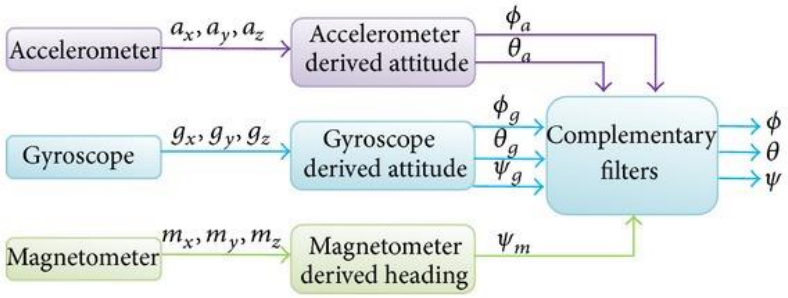
$\boldsymbol{\gamma}$ = 0.995 (Basic AHRS) k_p = 0.13, k_i = 0.3 (Mahony) β = 0.033 (Madgwick)

https://github.com/pololu/minimu-9-ahrs-arduino

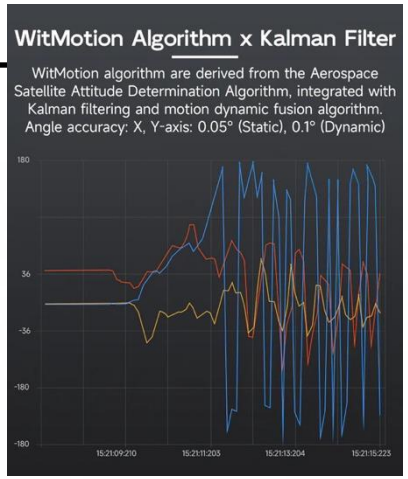
Complementary Filtering

$$angle = 0.98 * (angle + gyrData * dt) + 0.02 * (accData)$$

```
#define ACCELEROMETER_SENSITIVITY 8192.0
#define GYROSCOPE_SENSITIVITY 65.536

#define M_PI 3.14159265359

#define dt 0.01 // 10 ms sample rate!

void ComplementaryFilter(short accData[3], short gyrData[3], \
                         float *pitch, float *roll)
{
    float pitchAcc, rollAcc;

    // Integrate the gyroscope data -> int(angularSpeed) = angle
    // Angle around the X-axis
    *pitch += ((float)gyrData[0] / GYROSCOPE_SENSITIVITY) * dt;
    // Angle around the Y-axis
    *roll -= ((float)gyrData[1] / GYROSCOPE_SENSITIVITY) * dt;

    // Compensate for drift with accelerometer data if !bullshit
    // Sensitivity = -2 to 2 G at 16Bit -> 2G = 32768 && 0.5G = 8192
    int forceMagnitudeApprox = abs(accData[0]) + abs(accData[1]) + abs(accData[2]);
    if (forceMagnitudeApprox > 8192 && forceMagnitudeApprox < 32768)
    {
    // Turning around the X axis results in a vector on the Y-axis
        pitchAcc = atan2f((float)accData[1], (float)accData[2]) * 180 / M_PI;
        *pitch = *pitch * 0.98 + pitchAcc * 0.02;

    // Turning around the Y axis results in a vector on the X-axis
        rollAcc = atan2f((float)accData[0], (float)accData[2]) * 180 / M_PI;
        *roll = *roll * 0.98 + rollAcc * 0.02;
    }
}
```



Sensor Fusion



Performed on factory calibration equipment

Sensor misalignment calibration

Temperature calibration

Factory calibration parameters

Saved on sensor flash memory

LP Motion Sensor ME1

12mm

12mm

Gyroscope

Accelerometer

Magnetometer

IMUcore (customized extended Kalman filter)

Orientation

Lin. acceleration

Relative displacement

Output interface

Implemented on STM32 microcontroller

Options: i2c, SPI, Bluetooth (classic / low energy), CAN bus, RS232, RS485, USB, Ethernet

Performed automatically during run-time

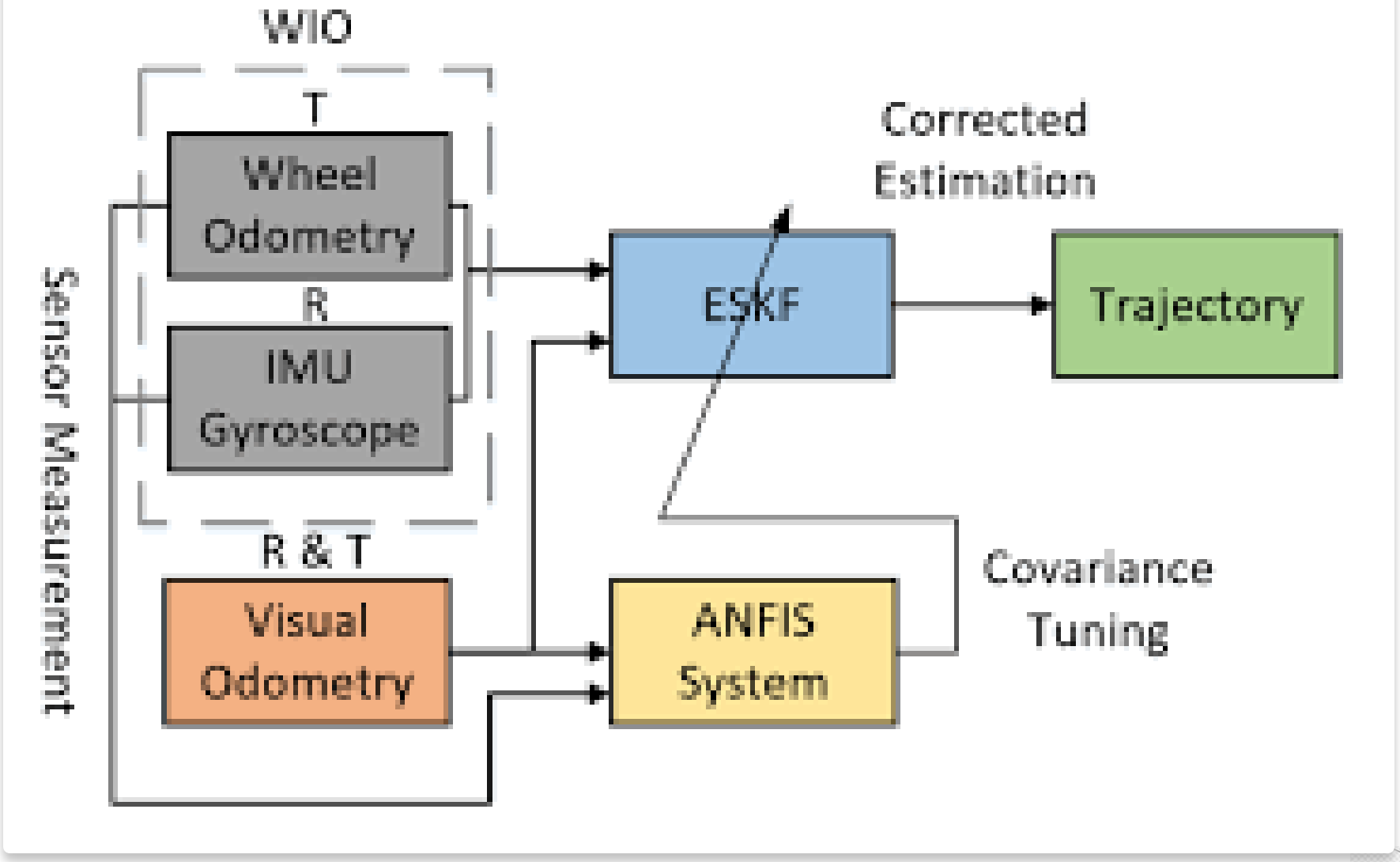
Gyroscope online calibration

Hard and soft iron magnetic field correction

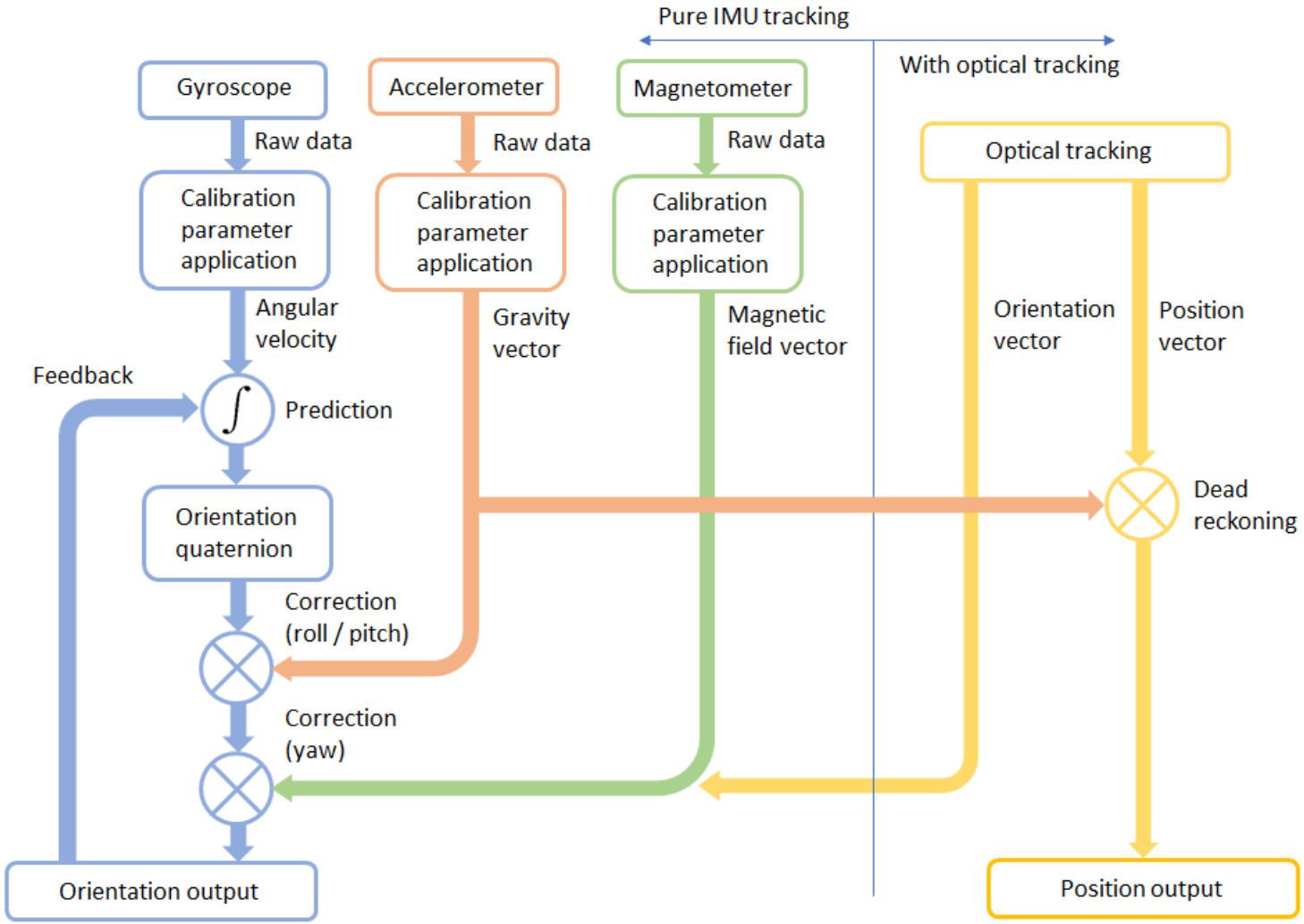
Magnetic noise rejection

Online calibration parameters

Odometry & IMU Fusion

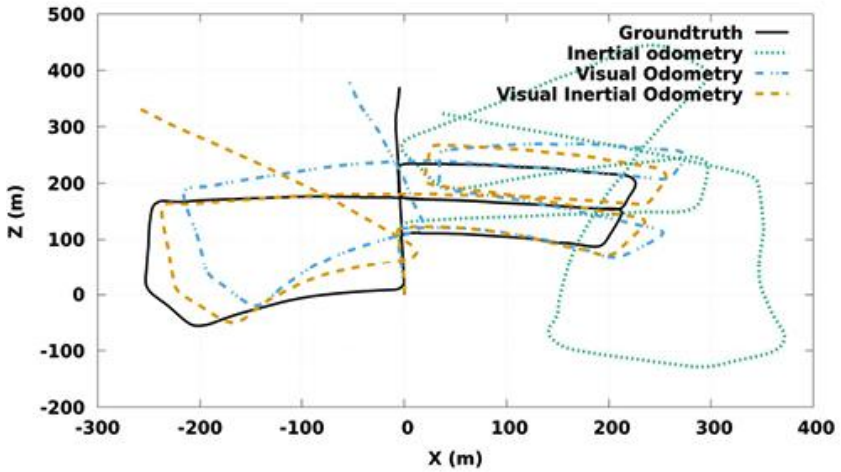


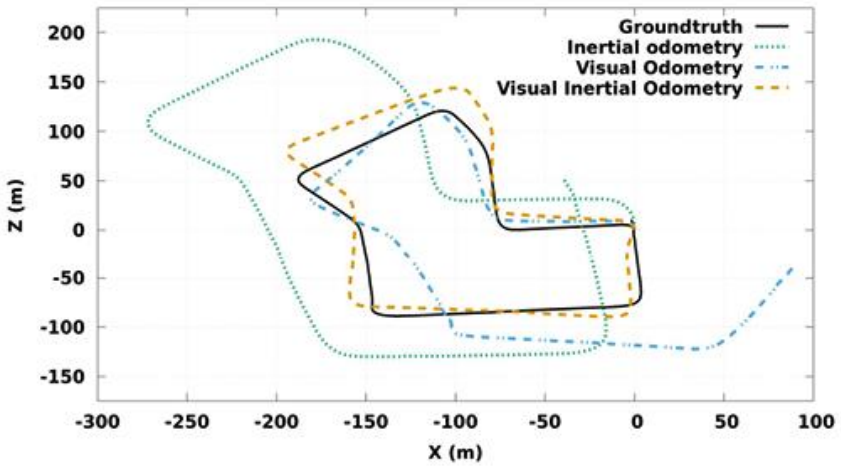
More Sensor Fusion



https://lp-research.com/imucore-sensor-fusion/

Result


Groundtruth
Inertial odometry
Visual Odometry
Visual Inertial Odometry
Z (m)
X (m)
500
400
300
200
100
0
-100
-200
-300
-200
-100
0
100
200
300
400


Groundtruth
Inertial odometry
Visual Odometry
Visual Inertial Odometry
Z (m)
X (m)
200
150
100
50
0
-50
-100
-150
-300
-250
-200
-150
-100
-50
0
50
100

ROS and Dead Reckoning

- ROS (Robot Operating System), dead reckoning is represented using several key components and messages that handle sensor data and compute the robot's pose over time.
- Odometry Message (nav_msgs/Odometry):
  - The odometry message is used to represent the robot’s estimated position (pose) and velocity in the world, as determined through dead reckoning.
  - Contains:
    - Pose: The robot's position (x, y, z) and orientation (usually as a quaternion).
    - Twist: The linear and angular velocity of the robot.
  - The message is continuously updated as the robot moves, based on wheel encoders or other motion sensors.

nav_msgs/Odometry.msg

nav_msgs/Odometry.msg

```
# This represents an estimate of a position and velocity in free space.
# The pose in this message should be specified in the coordinate frame given by header.frame_id.
# The twist in this message should be specified in the coordinate frame given by the child_frame_id
Header header
string child_frame_id
geometry_msgs/PoseWithCovariance pose
geometry_msgs/TwistWithCovariance twist
```

geometry_msgs/PoseWithCovariance.msg

```
# This represents a pose in free space with uncertainty.
Pose pose

# Row-major representation of the 6x6 covariance matrix
# The orientation parameters use a fixed-axis representation.
# In order, the parameters are:
# (x, y, z, rotation about X axis, rotation about Y axis, rotation about Z axis)
float64[36] covariance
```

geometry_msgs/TwistWithCovariance.msg

```
# This expresses velocity in free space with uncertainty.
Twist twist

# Row-major representation of the 6x6 covariance matrix
# The orientation parameters use a fixed-axis representation.
# In order, the parameters are:
# (x, y, z, rotation about X axis, rotation about Y axis, rotation about Z axis)
float64[36] covariance
```

Example

```
twist:  linear:    x: 0.5    y: 0.0    z: 0.0
angular:    x: 0.0    y: 0.0    z: 0.1
covariance: [0.02, 0, 0, 0, 0, 0,   # x velocity uncertainty
 0, 0.02, 0, 0, 0, 0,   # y velocity uncertainty
0, 0, 0.02, 0, 0, 0,   # z velocity uncertainty
0, 0, 0, 0.005, 0, 0,  # roll velocity uncertainty
0, 0, 0, 0, 0.005, 0,  # pitch velocity uncertainty
0, 0, 0, 0, 0, 0.005]  # yaw velocity uncertainty
```


Odom in software

- robot's odometry frame (often called odom) is used to maintain a running estimate of the robot’s position and orientation.

```
#include <ros/ros.h>
#include <tf/transform_broadcaster.h>
#include <nav_msgs/Odometry.h>

int main(int argc, char** argv){
  ros::init(argc, argv, "odometry_publisher");

  ros::NodeHandle n;
  ros::Publisher odom_pub = n.advertise<nav_msgs::Odometry>("odom", 50);
  tf::TransformBroadcaster odom_broadcaster;

  double x = 0.0;
  double y = 0.0;
  double th = 0.0;

  double vx = 0.1;
  double vy = -0.1;
  double vth = 0.1;

  ros::Time current_time, last_time;
  current_time = ros::Time::now();
  last_time = ros::Time::now();

  ros::Rate r(1.0);
```


```
while(n.ok()){
  current_time = ros::Time::now();

  //compute odometry in a typical way given the velocities of the robot
  double dt = (current_time - last_time).toSec();
  double delta_x = (vx * cos(th) - vy * sin(th)) * dt;
  double delta_y = (vx * sin(th) + vy * cos(th)) * dt;
  double delta_th = vth * dt;

  x += delta_x;
  y += delta_y;
  th += delta_th;

  //since all odometry is 6DOF we'll need a quaternion created from yaw
  geometry_msgs::Quaternion odom_quat = tf::createQuaternionMsgFromYaw(th);

  //first, we'll publish the transform over tf
  geometry_msgs::TransformStamped odom_trans;
  odom_trans.header.stamp = current_time;
  odom_trans.header.frame_id = "odom";
  odom_trans.child_frame_id = "base_link";

  odom_trans.transform.translation.x = x;
  odom_trans.transform.translation.y = y;
  odom_trans.transform.translation.z = 0.0;
  odom_trans.transform.rotation = odom_quat;

  //send the transform
  odom_broadcaster.sendTransform(odom_trans);
```


```
  //next, we'll publish the odometry message over ROS
  nav_msgs::Odometry odom;
  odom.header.stamp = current_time;
  odom.header.frame_id = "odom";
  odom.child_frame_id = "base_link";

  //set the position
  odom.pose.pose.position.x = x;
  odom.pose.pose.position.y = y;
  odom.pose.pose.position.z = 0.0;
  odom.pose.pose.orientation = odom_quat;

  //set the velocity
  odom.twist.twist.linear.x = vx;
  odom.twist.twist.linear.y = vy;
  odom.twist.twist.angular.z = vth;

  //publish the message
  odom_pub.publish(odom);

  last_time = current_time;
  r.sleep();
}
```


Odom in Hardware

- Wheel Encoders: Provide data on the distance traveled by each wheel, allowing the robot to estimate its movement (distance and orientation changes).
- Inertial Measurement Unit (IMU): Provides data on acceleration and angular velocity, which can be used to estimate changes in orientation and velocity.

- These sensors contribute to the dead reckoning calculations and are fused together to improve accuracy.

Will discuss more in the next lecture

Fusion Package

- Sensor Fusion (e.g., robot_localization package):
  - The robot_localization package is commonly used for fusing multiple sensor inputs (e.g., IMU, odometry, GPS) to improve dead reckoning estimates.
  - It uses algorithms like Extended Kalman Filters (EKF) to combine odometry, IMU, and other data sources to produce a more accurate estimate of the robot's pose.

Will discuss more in the following lecture